Ammeter Testing Framework - Design Documentation

Overview

The framework tests three ammeter emulators (Greenlee, ENTES, CIRCUTOR) over TCP: it samples each one on a fixed schedule, analyzes and plots the readings, compares the ammeters with each other, and archives every run so runs can be reviewed and compared later. Everything is driven by `config/config.yaml`.

```
config.yaml
  -> AmmeterTestFramework     (builds and validates everything from the config)
       -> ammeter_api         one interface for every ammeter
       -> sampler             fixed-schedule measurements
       -> analyzer, visualizer   statistics and plots per ammeter
       -> accuracy            compares the ammeters with each other
       -> result_store        archives the run as JSON
       -> report              prints it
```

Run it: `.venv/bin/python main.py [label]` tests everything and prints the report. `history.py list | show <id|latest> | compare <id> <id>` reviews archived runs. `python -m pytest tests` runs the tests (no emulators needed). Details are in the README.

Libraries installed: matplotlib 3.9.4 (plots) and PyYAML 6.0.3 (config). numpy came in as matplotlib's dependency. scipy, numpy, seaborn and pandas were installed at first, then removed from `requirements.txt` because nothing uses them. scipy was only needed for the t-distribution value in the confidence interval, which is now computed in pure Python (`src/utils/tdist.py`).

Fixes to the supplied code

Problem	Cause	Fix
`main.py` printed no data	The client calls were commented out and the commands truncated. The server answers only when it receives its full command.	Re-enabled the calls with the full commands (ports and commands now come from config).
"Address already in use" on a quick restart	Closed connections stay in TCP `TIME_WAIT` and the servers did not set `SO_REUSEADDR` .	Set it in `AmmeterEmulatorBase` before `bind()` , except on Windows, where it would allow two processes to share a live port and is not needed.
CIRCUTOR command differed from the README	The emulator required an undocumented `-current` suffix.	Removed the suffix so it matches the README and config.
`test_framework.py` crashed on import	`Dict` was used without being imported, and `run_test` was an empty stub.	Rewrote it on top of the new API.

Also: the README listed ports 5000-5002 while the code uses 5001-5003, so the README was rewritten to match. The dead `src/utils/logger.py` was deleted, and `Ammeters/client.py` is left as supplied (`main.py` no longer uses it).

Design decisions

1. One interface for every ammeter. `AmmeterInterface.measure()` handles timing, parsing and error reporting; a subclass implements only `_read_raw()` . Every call returns the same `MeasurementResult` (value, success, unit, timestamp, duration, error) and *never raises. Why:* failures are data. A dead ammeter must not stop a sampling run, and a new ammeter type or transport is one small class.

2. Configuration over code. Ports, commands, sampling, metrics, plots and expected currents live in `config.yaml` ; per-ammeter `enabled` , `sampling` and `error_rate` override the defaults. The config is validated when the framework is built, so a typo fails immediately with the list of valid names, not after a minute of sampling. *Why:* ENTES uses a different command than the others, and anything that differs belongs in data, not branches.

3. A fixed sampling schedule. Sample *i* is due at `start + i * interval`, measured from the start of the run, not from the previous sample, so measurement time never accumulates into drift. The actual time of every sample is recorded. *Why:* with 10 to 200 Hz runs, the worst deviation from the schedule was 0.6 to 2.7 ms, and the report shows it rather than assuming it.

4. Small, extendable analysis. Statistics use the standard library (`statistics`), with failed samples reported separately. Metrics and plots are registries (`METRICS`, `PLOTS`, `COMPARISON_PLOTS`): a new metric or plot is one function and one entry. *Why:* minimal dependencies, and extension without touching the framework.

5. Every run is archived, whole. One JSON file per run, named `<UTC timestamp>-<random>`, holding the raw samples, analysis, accuracy and a snapshot of the config. Files are written atomically and run ids are validated (no path tricks). *Why:* a run is reproducible and comparable months later, and a crash never leaves a half-written run.

6. Errors are explicit. Every deliberate error derives from `FrameworkError`; the entry points print `error: <message>` and exit 1 instead of a stack trace. A run where an ammeter gives no valid sample prints a `WARNING`, lists failures by kind in an `ERRORS` section, and exits 1. Error simulation (`error_rate`) lets this be exercised without breaking an emulator.

Accuracy assessment

The emulators do not measure a shared true current: each generates its own random value on its own scale (about 0.38 A, 69 A and 0.03 A), so comparing raw values across ammeters is meaningless. The assessment therefore uses two layers:

- **Precision, needing no reference:** coefficient of variation, a robust spread (IQR / median) and a 95% confidence interval of the mean (t distribution, correct for small samples, computed without scipy). All are scale-free, so the ammeters can be ranked against each other.
- **Accuracy against the documented model:** the README's input ranges give each emulator's expected mean exactly (for example ENTES 0.055 T x 1250 = 68.75 A). The report shows the bias in percent and whether the expected value lies inside the interval, which separates a real deviation from sampling noise.
- **Most reliable method:** the highest success rate first, then the lowest spread. In typical runs this is CIRCUTOR (CV about 0.5, and the smallest bias).

I checked the interval, spread and bias against scipy and numpy on known data, and the pure-Python t quantile against scipy over 120 combinations of degrees of freedom and probability (worst relative error 4e-11). A simulation of 2,000 runs found the expected mean inside the 95% interval 93.7% of the time.

Bonus challenges

- **Error simulation:** `error_rate` / `error_seed` on an ammeter makes that fraction of reads fail (reset, timeout, empty, garbage or non-finite reply).
- **Advanced visualization:** per ammeter, a rolling mean with a +/- 1 std dev band; per run, a box plot of readings normalized by each ammeter's own mean, and a bias chart with confidence intervals.
- **Configuration-driven testing:** design decision 2.

Limitations

- With 20 samples the ranking can change between runs, and Greenlee's readings (V/R) are heavy-tailed, so its mean and interval are unstable; use more measurements for a steadier verdict.
- Tested on macOS only; the Windows handling of `SO_REUSEADDR` is by design, not tested.
- Unit tests use a fake ammeter plus a throwaway local TCP server for real socket failures (closed port, hang-up, no answer); the real emulators are exercised by `main.py`.

More detail (per-module notes, measured figures) is in `docs/DETAILED_NOTES.md`.